

NoSQL Optimization

- It means designing data models, queries and infrastructure so the system performs efficiently at scale.
- NoSQL optimization is about :-
 - Data modeling
 - Access patterns
 - Partition strategy
 - Memory usage & Index design

Methods

1) Design based on Access patterns

In SQL $\rightarrow$ we normalize first

In NoSQL $\rightarrow$ Design based on how data is ~~required~~ ^{queried}.

EX:- If fetching orders is done by user-id frequently we design documents around user-id.

2) Proper Indexing $\rightarrow$ Create index on frequently queried fields

Without index $\rightarrow$ full collection scan occurs.

But too many index $\rightarrow$ slows down writes.

3) Avoid Large Documents $\rightarrow$ Split them into large nested structure

4) Choose Good Partition (Shard) Key (Read)

5) Denormalization $\rightarrow$ duplicating only necessary fields.

- Keeping frequently updated fields separate.

6) Caching :- Reduces DB pressure

Example

- Large e-commerce system using MongoDB:
 - Product catalog $\rightarrow$ Indexed by category & price
 - Orders $\rightarrow$ Partitioned by user-id.
 - Cache popular products in Redis.
 - Archive old orders.

Database Migration

- It is a controlled, versioned change to DB schema.
- Allows us to :-
 - Track schema changes
 - Revert them if needed
- Apply them safely
- Keep all environment in sync.

Why they are needed?

In real backend systems :- Multiple developers work together

- staging and production must watch
- code and database must evolve together.

Without migration

- one developer adds column locally
- Another developer's system breaks
- Production schema differs from development.

Migrations make schema changes reproducible & traceable.

Safe Migration strategies

- Avoid long locks
- Use backward-compatible changes.
- Deploy in phases
- Test on stages & monitor performance

Zero-Downtime Migration

- To achieve this
 - Avoid dropping columns immediately
 - Avoid renaming directly
 - Use additive changes first.

* Poor Migration handling causes :-

- Production outages
- Broken deployments
- Data loss
- System downtime.

Connection Pooling

- It means reusing a set of pre-opened database connections instead of creating new ones for every request.

Instead of :- open $\rightarrow$ query $\rightarrow$ Close (Multiple times)

We do :- Open 20 connections at startup and reuse them for all requests.

Working

- Imagine : Pool size = 10 connections. When request comes:
1) App asks pool for connection 2) Pool gives available connection
3) Query runs 4) Connection returned to pool.

Note :- connection is not closed, stays ready for next request.

Why not create very large pool?

- Too many connections $\rightarrow$ consume memory & overload DB
- Optimal pool size depends on :-
 - CPU cores
 - Query time
 - Database capacity

Advanced considerations

- 1) Idle Timeout $\rightarrow$ closed unused connections after some time
- 2) Connection leak $\rightarrow$ If app forgets to return connection to pool.
 - Pool exhausts
- 3) Read vs Write Pools $\rightarrow$ In system with replication :-
 - one pool for primary (writes)
 - one for replicas (reads)

Backend

- While designing system :
 - Always use connection pooling
 - Configure pool size carefully
 - Monitor active connections
 - Avoid connection leaks.